

Documento de Arquitetura do Software SARC (Sistema de Auxílio para Representantes Comerciais)

Alisson Rodrigues Fernandes¹

¹Instituto Federal de Educação, Ciência e Tecnologia da Bahia (IFBA)
Campus Vitória da Conquista

alissonrodrigues565656@gmail.com

Resumo. *Este artigo apresenta a definição arquitetural do SARC – Sistema de Auxílio para Representantes Comerciais. Este documento foi criado a partir do documento de requisitos de software do referido sistema. São discutidas as decisões arquiteturais adotadas e suas consequências, tendo sido adotado, como formato de registro de tais decisões, Architecture Decision Records (ADRs).*

1. Introdução

Este trabalho apresenta a definição arquitetural do SARC (Sistema de Auxílio para Representantes Comerciais), um sistema voltado à automatização da rotina de representantes comerciais, facilitando a geração de pedidos, o gerenciamento de fabricantes, clientes e produtos, além da emissão de relatórios e controle de backups. A arquitetura é derivada a partir dos requisitos do sistema, considerando restrições de segurança, disponibilidade, atômica das transações e necessidades específicas do domínio de representação comercial.

O SARC possui dois perfis de usuários: o **Representante Comercial**, que é o usuário principal com acesso completo a todas as funcionalidades do sistema, e o **Vendedor**, responsável apenas pelas funções de venda e consulta de relatórios de pedidos. Essa distinção de papéis influencia diretamente as decisões arquiteturais relacionadas ao controle de acesso e segurança.

O objetivo deste documento é registrar as decisões arquiteturais tomadas para o SARC, justificando-as com base nos requisitos levantados e analisando suas consequências para o desenvolvimento e evolução do sistema.

2. Resumo dos Requisitos

Os principais Requisitos Funcionais (RF) do sistema incluem:

- **RQF001:** Cadastrar fabricantes (CRUD com pesquisa por CNPJ e razão social)
- **RQF002:** Cadastrar produtos (CRUD com pesquisa por código e descrição)
- **RQF003:** Cadastrar clientes (CRUD com pesquisa por CNPJ e razão social)
- **RQF004:** Cadastrar pedidos (gerenciamento de pedidos com código automático)
- **RQF005:** Cadastrar vendedor (gerenciamento de vendedores com autenticação)
- **RQF006:** Visualizar pedido (exibição detalhada de informações do pedido)
- **RQF007:** Imprimir relatório mensal/anual dos produtos mais vendidos
- **RQF008:** Imprimir lista de produtos cadastrados
- **RQF009:** Imprimir relatório de produtos por fabricante

- **RQF010:** Imprimir relatório de pedidos por fabricante
- **RQF011:** Gerar arquivo de backup (exportação da base de dados)
- **RQF012:** Importar arquivo de backup (restauração da base de dados)

Requisitos Não-Funcionais relevantes:

- **RNF001:** Linguagem de programação Java
- **RNF002:** Atomicidade nas transações de pedidos
- **RNF003:** Segurança com controle de privilégios por perfil de usuário
- **RNF004:** Disponibilidade mínima de 98% em dias úteis e 80% em fins de semana
- **RNF005:** Segurança dos backups com exportações periódicas

3. Proposta Arquitetural

A arquitetura adotada, exibida na Figura 1, segue, como estilo arquitetural, a **Arquitetura em N-Camadas (Layered Architecture)**, organizando o sistema em três camadas principais: Apresentação, Lógica de Negócio e Acesso a Dados. Este estilo foi escolhido por ser o mais adequado para sistemas empresariais que exigem separação clara de responsabilidades, facilitando a manutenção, testabilidade e evolução do sistema. A escolha da Arquitetura em N-Camadas se justifica pelas seguintes razões:

1. **Natureza do sistema:** O SARC é um sistema de gestão empresarial com operações CRUD intensivas e geração de relatórios, cenário ideal para camadas bem definidas;
2. **Separação de responsabilidades:** Cada camada possui uma responsabilidade única, permitindo que mudanças em uma camada (ex.: alteração na interface) não afetem outras (ex.: regras de negócio);
3. **Facilidade de manutenção:** A organização em camadas permite que desenvolvedores trabalhem em partes isoladas do sistema;
4. **Conformidade com Java:** O ecossistema Java possui amplo suporte a padrões de camadas, com frameworks consolidados para cada nível.

Os principais componentes incluem:

- **Camada de Apresentação (front-end):** responsável pela interface gráfica do sistema, exibindo formulários de cadastro, telas de pesquisa, visualização de pedidos e relatórios. Interage diretamente com o usuário (Representante ou Vendedor);
- **Camada de Lógica de Negócio (back-end):** contém as regras de negócio do sistema, como validações de CNPJ, controle de atomicidade dos pedidos, geração de relatórios, gerenciamento de permissões por perfil de usuário e controle de backup;
- **Camada de Acesso a Dados (persistência):** responsável pela comunicação com o banco de dados relacional, realizando operações de persistência (CRUD), consultas para relatórios e transações atômicas;
- **Banco de Dados Relacional:** armazena e disponibiliza os dados de fabricantes, produtos, clientes, pedidos, vendedores e backups;
- **Sistema de Arquivos:** utilizado para armazenamento dos arquivos de backup exportados e relatórios gerados.

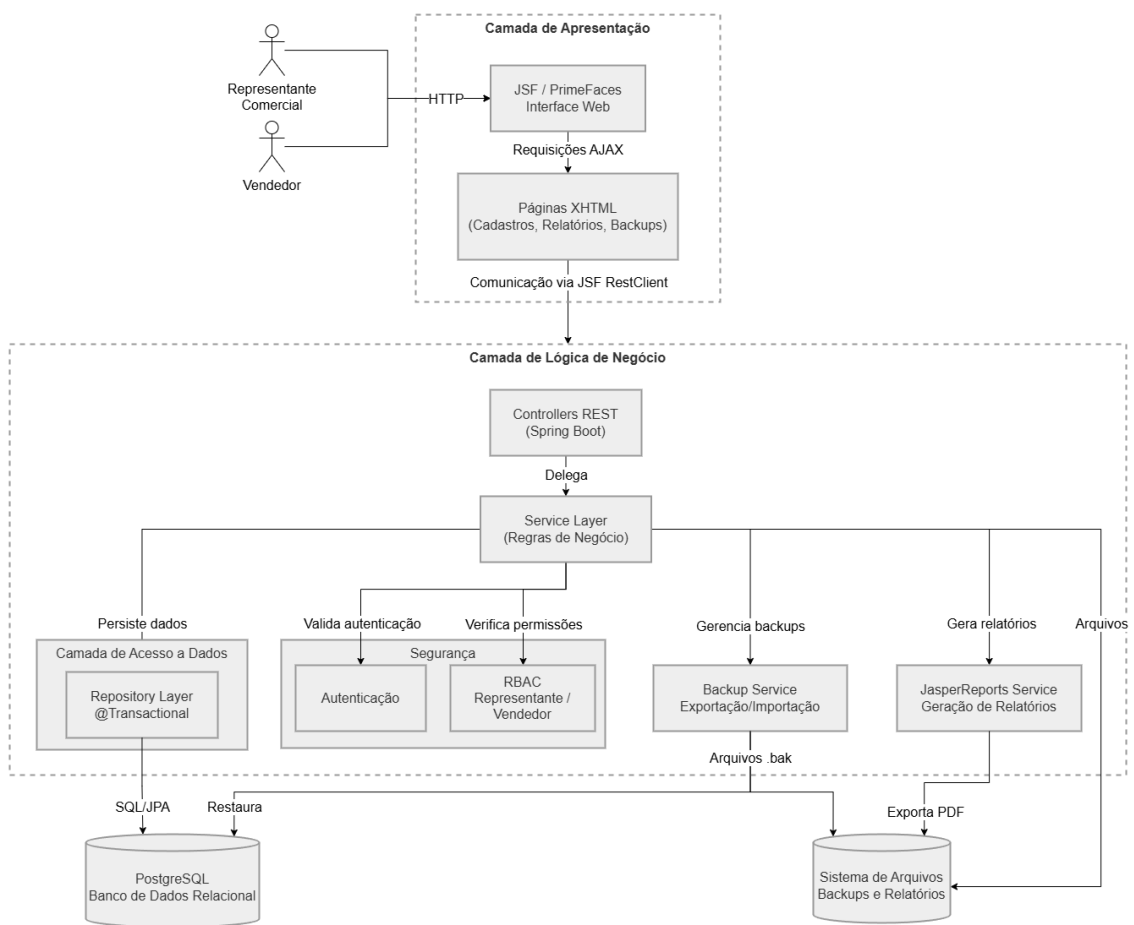


Figura 1. Visão geral da arquitetura do SARC. As camadas são: Apresentação (JSF/PrimeFaces), Lógica de Negócio (Spring Boot com RBAC e transações), Acesso a Dados (JPA/Hibernate) e Persistência (PostgreSQL e sistema de arquivos).

4. Decisões Arquiteturais

Considerando os requisitos apresentados na Seção 2, abordamos as decisões arquiteturais que devem ser implementadas. Cada uma das subseções seguintes trata de uma decisão, registrada no formato de Architecture Decision Records (ADR).

4.1. ADR-01: Arquitetura em N-Camadas (Layered Architecture)

Status: Aceita

Contexto:

O SARC é um sistema de gestão empresarial com múltiplas entidades (fabricantes, produtos, clientes, pedidos, vendedores), operações CRUD intensivas, geração de relatórios e dois perfis de usuários com diferentes níveis de acesso. É necessário um estilo arquitetural que promova organização, manutenibilidade e separação de responsabilidades.

Decisão:

Adotar a Arquitetura em N-Camadas (3 camadas: Apresentação, Lógica de Negócio e Acesso a Dados), onde cada camada possui responsabilidades bem definidas e se comunica apenas com as camadas adjacentes.

Consequências:

- **Positiva:** Separação clara de responsabilidades, facilitando a manutenção e evolução independente de cada camada;
- **Positiva:** Maior testabilidade, pois cada camada pode ser testada isoladamente;
- **Negativa:** Overhead de comunicação entre camadas, podendo impactar ligeiramente o desempenho;
- **Negativa:** Rigidez na estrutura, dificultando cenários que exigem comunicação direta entre camadas não adjacentes.

4.2. ADR-02: Desenvolvimento em Java com Monorepo (Backend Spring Boot + Frontend JSF)

Status: Aceita

Contexto:

O requisito não-funcional RNF001 determina que o sistema deve ser desenvolvido em Java. É necessário escolher tecnologias mais atuais e adequadas para construir a aplicação.

Decisão:

Desenvolver o sistema em um **monorepo** contendo dois módulos principais:

- **Backend:** Utilizando Java com Spring Boot, expondo uma API RESTful. Responsável pelas regras de negócio, persistência e segurança.
- **Frontend:** Utilizando JavaServer Faces (JSF) com componentes PrimeFaces para a camada de apresentação web. O frontend consome a API do backend.

Para a camada de persistência, será utilizado JPA/Hibernate¹.

Consequências:

¹<https://hibernate.org/>

- **Positiva:** Backend e frontend em Java, aproveitando a portabilidade entre sistemas operacionais (Windows, Linux, macOS) graças à JVM;
- **Positiva:** Ecossistema maduro do Spring Boot e JSF com grande comunidade e vasta documentação;
- **Positiva:** Monorepo facilita o gerenciamento de código, versionamento e pipelines de CI/CD;
- **Negativa:** Maior consumo de memória em comparação com frontends nativos ou leves;
- **Negativa:** Curva de aprendizado para desenvolvedores não familiarizados com JSF ou Spring Boot;
- **Negativa:** Comunicação rede entre frontend e backend pode adicionar latência.

4.3. ADR-03: Controle de Acesso Baseado em Papéis (RBAC)

Status: Aceita

Contexto:

O sistema possui dois perfis de usuários com permissões distintas: o Representante Comercial (acesso total) e o Vendedor (acesso restrito a vendas e consultas). O RNF003 exige que somente usuários com privilégios administrativos possam gerenciar todas as informações do sistema, enquanto o RQF005 define o cadastro de vendedores com credenciais de acesso.

Decisão:

Implementar o padrão de Controle de Acesso Baseado em Papéis ou, em inglês, Role-Based Access Control (RBAC), utilizando Spring Security com autenticação baseada em sessão HTTP. Após a autenticação, o servidor manterá a sessão do usuário autenticado, associando suas permissões e papel (Representante ou Vendedor) durante toda a navegação no sistema.

As permissões serão controladas através de roles associadas aos usuários, permitindo restringir funcionalidades administrativas apenas ao Representante Comercial.

Consequências:

- **Positiva:** Integração natural com aplicações JSF/PrimeFaces baseadas em renderização server-side;
- **Positiva:** Simplificação da autenticação e gerenciamento de sessão;
- **Positiva:** Controle granular de permissões por perfil de usuário;
- **Negativa:** Dependência de sessões no servidor, reduzindo a escalabilidade horizontal em comparação com abordagens stateless;
- **Negativa:** Necessidade de gerenciamento de expiração e invalidação de sessões.

4.4. ADR-04: Gerenciamento Transacional de Pedidos

Status: Aceita

Contexto:

O RQF004 define o cadastro e gerenciamento de pedidos, que envolvem múltiplas entidades (fabricante, cliente, produtos). O RNF002 exige que todas as ações do pedido sejam realizadas de forma atômica: ou todas as operações são executadas com sucesso, ou nenhuma delas é persistida. Além disso, múltiplos vendedores podem realizar operações

simultaneamente sobre pedidos e produtos, tornando necessário um mecanismo de controle de concorrência para evitar inconsistências causadas por atualizações concorrentes.

Decisão:

Utilizar o gerenciamento transacional do banco de dados relacional, implementado através do Spring Boot com anotações `@Transactional` e JPA/Hibernate, garantindo atomicidade, consistência, isolamento e durabilidade nas operações de pedidos.

Para cenários de concorrência, o sistema poderá utilizar controle de concorrência otimista através de versionamento de entidades com `@Version`, evitando sobrescrita acidental de dados em atualizações simultâneas.

Consequências:

- **Positiva:** Garantia de consistência dos dados, evitando estados parciais em caso de falhas;
- **Positiva:** Conformidade com o requisito de atomicidade (RNF002);
- **Positiva:** Redução do risco de inconsistências causadas por alterações concorrentes em pedidos e produtos;
- **Negativa:** Potencial para deadlocks em cenários de alta concorrência;
- **Negativa:** Redução de desempenho em operações que envolvem muitos produtos em um único pedido.

4.5. ADR-05: Banco de Dados Relacional

Status: Aceita

Contexto:

O SARC gerencia múltiplas entidades com relacionamentos complexos: fabricantes possuem produtos, clientes realizam pedidos, pedidos contêm produtos de fabricantes específicos (RQF001, RQF002, RQF003). É necessário um mecanismo de persistência que suporte integridade referencial e consultas complexas para relatórios.

Decisão:

Adotar um banco de dados relacional (PostgreSQL²) para persistência dos dados, aproveitando recursos de integridade referencial, índices e suporte nativo a transações ACID.

Consequências:

- **Positiva:** Integridade referencial entre entidades (fabricantes, produtos, clientes, pedidos), evitando dados inconsistentes;
- **Positiva:** Suporte robusto a consultas complexas necessárias para os relatórios (RQF007 a RQF010);
- **Negativa:** Rigidez do esquema, exigindo migrações para alterações estruturais;
- **Negativa:** Limitações de escalabilidade horizontal em comparação com bancos NoSQL.

²<https://www.postgresql.org/>

4.6. ADR-06: Motor de Geração de Relatórios

Status: Aceita

Contexto:

Os requisitos RQF007, RQF008, RQF009 e RQF010 exigem a geração e impressão de diferentes tipos de relatórios: relatório mensal/anual de produtos mais vendidos, lista de produtos cadastrados, relatório de produtos por fabricante e relatório de pedidos por fabricante. Esses relatórios devem ser formatados profissionalmente e suportar impressão.

Decisão:

Utilizar o framework JasperReports³ para geração de relatórios, permitindo a criação de templates reutilizáveis em formato JRXML e exportação para PDF e outros formatos.

Consequências:

- **Positiva:** Geração de relatórios profissionais com formatação avançada e suporte a impressão direta;
- **Positiva:** Templates reutilizáveis e parametrizáveis que facilitam a criação de novos relatórios;
- **Negativa:** Curva de aprendizado para configuração dos templates JRXML;
- **Negativa:** Dependência de uma biblioteca externa que pode aumentar o tamanho do artefato final.

4.7. ADR-07: Estratégia de Backup e Recuperação

Status: Aceita

Contexto:

Os requisitos RQF011 e RQF012 demandam a exportação e importação de backups da base de dados. O RNF005 complementa determinando que exportações periódicas devem ser realizadas e os arquivos gerados devem ser armazenados em locais seguros. É necessário garantir que backups corrompidos não sejam importados.

Decisão:

Implementar um mecanismo de backup utilizando ferramentas nativas do SGBD (ex.: *pg_dump*), com os arquivos armazenados no sistema de arquivos local e possibilidade de cópia para armazenamento externo.

Consequências:

- **Positiva:** Proteção contra perda de dados através da realização periódica de backups da base de dados;
- **Positiva:** Utilização de ferramentas nativas do PostgreSQL aumenta a confiabilidade e compatibilidade do processo de restauração;
- **Positiva:** Verificações de integridade dos arquivos de backup reduzem o risco de importação de dados corrompidos;
- **Negativa:** O armazenamento contínuo de backups pode aumentar significativamente o consumo de espaço em disco;

³<https://community.jaspersoft.com/projects/jasperreports-library/>

- **Negativa:** O processo de exportação e restauração pode impactar o desempenho do sistema durante operações de backup em horários de pico;
- **Negativa:** Dependência do sistema de arquivos local pode representar um ponto único de falha caso não exista cópia externa dos backups.

4.8. ADR-08: Alta Disponibilidade

Status: Aceita

Contexto:

O RNF004 exige que o sistema esteja disponível em, no mínimo, 98% do tempo durante a semana e 80% em fins de semana e feriados. Dado que representantes comerciais atuam em horários flexíveis, o sistema deve permanecer operacional durante a maior parte do tempo, reduzindo indisponibilidades causadas por falhas de software ou hardware.

Decisão:

Implementar mecanismos de monitoramento e recuperação automática da aplicação, utilizando serviços de supervisão de processos e rotinas periódicas de verificação de disponibilidade. O sistema também deverá possuir políticas de backup periódico e possibilidade de restauração rápida em caso de falhas críticas.

Além disso, a aplicação será implantada em ambiente dedicado com monitoramento de recursos computacionais, permitindo identificar falhas e reiniciar serviços automaticamente quando necessário.

Consequências:

- **Positiva:** Redução do tempo de indisponibilidade do sistema através da reinicialização automática de serviços;
- **Positiva:** Maior confiabilidade operacional com monitoramento contínuo da aplicação;
- **Positiva:** Possibilidade de recuperação mais rápida em cenários de falha crítica através dos backups periódicos;
- **Negativa:** Necessidade de configuração e manutenção adicional dos mecanismos de monitoramento;
- **Negativa:** Custos adicionais relacionados à infraestrutura dedicada e armazenamento de backups;
- **Negativa:** O reinício automático de serviços pode não resolver falhas causadas por erros lógicos na aplicação.

4.9. ADR-09: Cache para Consultas e Relatórios

Status: Em discussão

Contexto:

Os relatórios de produtos mais vendidos, produtos por fabricante e pedidos por fabricante (RQF007 a RQF010) podem executar consultas complexas envolvendo grandes volumes de dados. À medida que o número de pedidos aumenta, essas consultas podem impactar o tempo de resposta da aplicação e aumentar a carga sobre o banco de dados.

Decisão:

Implementar um mecanismo de cache utilizando Spring Cache com provedor Redis para

armazenar temporariamente os resultados de consultas frequentemente acessadas, especialmente relatórios e dados de referência que sofrem poucas alterações.

A invalidação do cache ocorrerá automaticamente sempre que operações de escrita afetarem os dados relacionados aos relatórios armazenados.

Consequências:

- **Positiva:** Redução do tempo de resposta para consultas repetidas;
- **Positiva:** Diminuição da carga sobre o banco de dados relacional;
- **Positiva:** Melhor experiência para os usuários durante a geração de relatórios;
- **Positiva:** Solução compatível com o ecossistema Spring Boot e facilmente expansível;
- **Negativa:** Aumento da complexidade arquitetural devido ao gerenciamento de invalidação de cache;
- **Negativa:** Necessidade de infraestrutura adicional para execução do Redis;
- **Negativa:** Possibilidade de exibição temporária de dados desatualizados caso ocorram falhas no processo de invalidação.

5. Ferramentas de Apoio

Durante o desenvolvimento deste trabalho foram utilizadas ferramentas de apoio para auxiliar na elaboração da arquitetura e na automatização de atividades relacionadas à documentação.

5.1. Código-fonte das Ferramentas MCP

O código-fonte das ferramentas MCP (Model Context Protocol) desenvolvidas para apoiar a elaboração e manutenção da documentação arquitetural encontra-se disponível no seguinte repositório:

- **Repositório do código-fonte:** <https://github.com/alissonfr/mcp-arquitetura-de-software>

5.2. Vídeo de Demonstração

Uma demonstração das funcionalidades das ferramentas MCP pode ser acessada por meio do vídeo disponível no link abaixo:

- **Vídeo de demonstração:** <https://drive.google.com/SEU-LINK-DO-VIDEO>

5.3. Outras Ferramentas Utilizadas

- **Inteligência Artificial (IA):** utilizada como ferramenta de apoio na análise dos requisitos, na estruturação do documento arquitetural e na elaboração das ADRs. Todas as sugestões geradas foram revisadas e validadas pelo autor.
- **Overleaf:** utilizado para edição, compilação e visualização do documento em $\text{\LaTeX}$, permitindo maior produtividade durante a elaboração da documentação.

5.4. Limitações

A utilização de ferramentas de IA como apoio ao desenvolvimento da documentação não substitui a análise crítica humana. Dessa forma, todas as decisões arquiteturais registradas neste documento foram revisadas pelo autor para garantir sua aderência aos requisitos e ao contexto do sistema.

6. Análise Crítica

A arquitetura proposta atende aos requisitos principais do sistema SARC, proporcionando uma estrutura organizada e manutenível para um sistema de gestão de representação comercial. Porém, apresenta limitações e trade-offs que merecem atenção.

A arquitetura atende aos requisitos ao organizar o sistema em camadas bem definidas que suportam todas as operações CRUD (RQF001–RQF005), os relatórios (RQF007–RQF010), o backup/recuperação (RQF011–RQF012) e os requisitos não-funcionais de segurança (RNF003), atômidade (RNF002) e disponibilidade (RNF004).

Entre os principais trade-offs estão:

- **Manutenibilidade vs. Desempenho:** A Arquitetura em N-Camadas promove excelente separação de responsabilidades, mas introduz overhead de comunicação entre camadas que pode impactar o desempenho em operações intensivas, como a geração de relatórios complexos;
- **Segurança vs. Usabilidade:** O RBAC garante controle rigoroso de acesso, mas adiciona complexidade no gerenciamento de sessões e permissões, podendo impactar a experiência do usuário caso ocorram expirações frequentes de sessão.
- **Consistência vs. Concorrência:** O gerenciamento transacional com ACID garante integridade total dos dados, mas pode causar gargalos em cenários com múltiplos vendedores realizando pedidos simultaneamente
- **Disponibilidade vs. Custo:** A infraestrutura redundante garante os níveis de disponibilidade exigidos, porém com custo operacional significativo

Melhorias futuras incluem:

- Introdução de mecanismo de cache para otimizar consultas frequentes de relatórios;
- Implementação de notificações em tempo real para acompanhamento de pedidos;
- Migração da camada de apresentação para um SPA moderno (React, Angular ou Vue), mantendo o backend Spring Boot;
- Automatização completa dos backups com armazenamento em nuvem;
- Introdução de dashboards analíticos para apoio à tomada de decisão comercial.

7. Conclusão

A arquitetura definida para o SARC atende aos requisitos propostos no documento de requisitos de software, equilibrando organização, segurança e confiabilidade. A adoção da Arquitetura em N-Camadas proporciona uma base sólida para a evolução do sistema, enquanto as decisões arquiteturais explícitas, registradas através de ADRs, permitem maior clareza sobre as escolhas realizadas e facilitam futuras evoluções do sistema.

O uso de tecnologias consolidadas do ecossistema Java (Spring Boot, JSF, JPA/Hibernate, JasperReports, PostgreSQL) garante maturidade e suporte adequado para o desenvolvimento e manutenção do sistema a longo prazo. Como direções futuras, destaca-se a possibilidade de migração para uma arquitetura web mais moderna e a incorporação de recursos analíticos avançados para apoiar a tomada de decisão dos representantes comerciais.